

COMP3308/3608 Week 5 - Introduction to Machine Learning, *k*-Nearest Neighbour, 1R

Author
K. Yu
2026

Contents

1	What is Machine Learning?	2
1.1	A taxonomy of ML	2
1.2	Supervised learning, formally	2
1.3	Unsupervised learning (clustering)	3
1.4	Reinforcement & association learning	3
1.5	ML vs Data Mining	3
2	The classification setup	3
2.1	Attribute types	3
3	<i>k</i>-Nearest Neighbour	4
3.1	Distance measures	4
3.2	The <i>k</i> -NN decision rules	4
3.3	Feature scaling	5
3.4	Distance with nominal attributes	6
3.5	Missing values (for distance-based methods)	6
3.6	Distance-weighted <i>k</i> -NN	6
3.7	Lazy vs eager learning; complexity	6
3.8	Curse of dimensionality	7
3.9	Decision boundary — Voronoi tessellation	7
3.10	Strengths and weaknesses	7
4	Rule Learning: the 1R Algorithm	8
4.1	Two design questions	8
4.2	1R on the weather data	9
4.3	Pseudocode and components	9
4.4	Handling missing values (generic, not just 1R)	10
4.5	Numeric attributes: discretisation	10
4.6	Overfitting and a better discretisation	10
4.7	Decision boundary of 1R	11
4.8	1R — discussion (Holte, 1993)	11

1 What is Machine Learning?

Machine Learning (ML) is the subfield of AI concerned with programs that improve from **examples**, **domain knowledge**, or **user feedback**. Dictionary-style definitions ("to gain knowledge", "to commit to memory") are inadequate for machines: they are either untestable (awareness, consciousness) or too passive (memorisation $\neq$ ability to generalise).

The operational definition we adopt (Herbert Simon):

Learning denotes changes in a system that are **adaptive**, enabling it to perform the same task — or tasks drawn from the same population — **more efficiently and more effectively** next time.

The key shift: learning is tied to **performance on a task**, not to the possession of knowledge. This is exactly the framing that makes ML measurable: we will quantify "better next time" with a loss/accuracy on unseen data.

1.1 A taxonomy of ML

Type	What is given	Goal
Supervised	Labelled $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$.	Learn $f : \mathbb{R}^d \rightarrow \mathcal{Y}$ that predicts y from $\mathbf{x}$.
Unsupervised	Only inputs $\{\mathbf{x}^{(i)}\}_{i=1}^n$.	Discover structure, e.g. clusters.
Reinforcement	Inputs plus a scalar <i>reward/grade</i> per action.	Learn a policy maximising long-run reward.
Association	Transactions or sequences.	Find co-occurrence or temporal patterns.

1.2 Supervised learning, formally

Given $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$ assumed drawn i.i.d. from an unknown joint distribution $p(\mathbf{x}, y)$, learn a function (a **classifier/regressor**)

$$f : \mathbb{R}^d \rightarrow \mathcal{Y}, \quad \hat{y} = f(\mathbf{x}),$$

that encapsulates the input-output relationship and **generalises** to new $\mathbf{x}$. "Supervised" because each training input comes with the *correct answer*⁽ⁱ⁾, which supervises the learner.

Two sub-cases, distinguished **only** by the type of the target variable:

- **Classification:** $\mathcal{Y}$ is a finite, pre-specified set (nominal, categorical), e.g. $\mathcal{Y} = \{\text{yes, no}\}$ or the 10 digit classes.
- **Regression:** $\mathcal{Y} = \mathbb{R}$ (numeric), e.g. predicting an exchange rate or CPU performance.

Examples of supervised algorithms: 1R, k -NN, decision trees, Naive Bayes, neural networks (perceptron, backprop, deep), SVM.

1.3 Unsupervised learning (clustering)

Given only $\{\mathbf{x}^{(i)}\}_{i=1}^n$ (no y), partition the data into clusters $C_1, \dots, C_K$ so that within-cluster points are similar and between-cluster points are dissimilar — informally, minimise within-cluster distance and maximise between-cluster distance. Algorithms: k -means, hierarchical clustering, nearest-neighbour clustering, DBSCAN, probabilistic clustering (Gaussian Mixture), etc.

1.4 Reinforcement & association learning

- **Reinforcement learning:** each experience carries a *score* rather than a correct label; most natural for control problems. Less common than supervised learning in this course.
- **Association learning:** market-basket analysis (items frequently bought together $\rightarrow$ e.g. cheese/jam with bread) and sequential analysis (frequent ordered patterns, e.g. web-navigation paths $A \rightarrow B \rightarrow C$, or DNA subsequences).

1.5 ML vs Data Mining

Data Mining = search **large** datasets for **meaningful, useful, actionable** hidden patterns. Most DM techniques originate in ML; DM is essentially *applied ML* operating at scale on large, high-dimensional data.

Motivation: the data explosion ("drowning in data but starving for knowledge").

Why ML matters: some tasks are definable only by examples; expert knowledge is too vast to hand-code; environments drift over time (e.g. spam evolves), so systems must adapt; and correlations hide inside big data.

2 The classification setup

- **Training data** $\mathcal{D}_{\text{train}}$: used to build the model.
- **Test data** $\mathcal{D}_{\text{test}}$: labelled, **disjoint** from training, used to estimate generalisation.
- **Accuracy** on a test set:

$$\text{Acc} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(\mathbf{x}, y) \in \mathcal{D}_{\text{test}}} \mathbf{1}\{f(\mathbf{x}) = y\},$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. Measuring on held-out data is what separates *memorisation* from *learning*.

2.1 Attribute types

- **Numeric (continuous):** values are numbers (e.g. *sepal length* = 5.1) Distances are meaningful directly.

- **Nominal (categorical):** values lie in a finite set (e.g. $outlook \in \{\text{sunny, overcast, rainy}\}$).
No intrinsic ordering or metric.

This distinction drives everything in k -NN (how to measure distance) and 1R (whether discretisation is needed).

3 k -Nearest Neighbour

Idea. Store all of $\mathcal{D}$. To classify an unseen input $\mathbf{x}$, find the closest training example(s) under a distance measure and return their label(s). Nothing is "fit" in advance.

3.1 Distance measures

Let $\mathbf{a} = [a_1 \ \cdots \ a_d]^\top$, $\mathbf{b} = [b_1 \ \cdots \ b_d]^\top$.

Euclidean (L_2):

$$d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2 = \sqrt{\sum_{j=1}^d (a_j - b_j)^2}.$$

Manhattan (L_1):

$$d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_1 = \sum_{j=1}^d |a_j - b_j|.$$

Minkowski (L_q , $q \in \mathbb{Z}^+$):

$$d_q(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_q = \left(\sum_{j=1}^d |a_j - b_j|^q \right)^{1/q}.$$

Special cases:

$$q = 2 \implies \text{Euclidean}, \quad q = 1 \implies \text{Manhattan}, \quad q \rightarrow \infty \implies \|\mathbf{a} - \mathbf{b}\|_\infty = \max\{a_j - b_j\}_{j=1}^d.$$

Aside (depth). A valid distance/metric satisfies non-negativity, identity of indiscernibles, symmetry, and the triangle inequality; all Minkowski distances with $q \geq 1$ are genuine metrics induced by the L_q norm $\|\mathbf{a} - \mathbf{b}\|_q$.

3.2 The k -NN decision rules

Let $N_k(\mathbf{x})$ be the index set of the k training points closest to $\mathbf{x}$.

Classification (majority vote):

$$\hat{y} = \operatorname{argmax}_{c \in \mathcal{Y}} \sum_{i \in N_k(\mathbf{x})} \mathbf{1}\{y^{(i)} = c\}.$$

Regression (average of neighbours):

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y^{(i)}$$

Larger k increases robustness to noise (one bad neighbour cannot dominate) at a small extra computational cost, but too large a k over-smooths and erases local structure. A common heuristic is to choose $k \approx \sqrt{n}$.

3.3 Feature scaling

Attributes on different scales let the large-scale attribute dominate the distance. Example: predict petrol consumption from *car weight* [kg] and *number of cylinders*.

$$\mathbf{x}^{(1)} = \begin{bmatrix} 6000 \\ 4 \end{bmatrix}, \quad \mathbf{x}^{(2)} = \begin{bmatrix} 1000 \\ 2 \end{bmatrix} \implies d_1(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}) = |6000 - 1000| + |4 - 2| = 5002,$$

so the cylinder feature is effectively invisible.

Min-max normalisation maps each attribute to $[0, 1]$:

$$\tilde{x}_j^{(i)} = \frac{x_j^{(i)} - \min\{x_j^{(i)}\}_{i=1}^n}{\max\{x_j^{(i)}\}_{i=1}^n - \min\{x_j^{(i)}\}_{i=1}^n}.$$

After normalising the example above, both features contribute comparably (e.g. $d_1(\tilde{\mathbf{x}}^{(1)}, \tilde{\mathbf{x}}^{(2)}) = |0 - 0.55| + |0 - 0.5| = 1.05$) and car weight no longer dominates.

Standardisation (Not Examinable). An alternative rescaling centres each feature at mean 0 with unit variance:

$$\tilde{x}_j^{(i)} = \frac{x_j^{(i)} - \mu_j}{\sigma_j}, \quad \mu_j = \frac{1}{n} \sum_{i=1}^n x_j^{(i)}, \quad \sigma_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_j^{(i)} - \mu_j)^2}.$$

Differences from min-max:

- Output range is unbounded (typically most values fall in $[-3, 3]$), not forced into $[0, 1]$.
- Far less sensitive to **outliers**: a single extreme value blows up $\max - \min$ and crushes everyone else under min-max, whereas it only mildly shifts μ_j, σ_j .
- Preserves the *shape* of the distribution; appropriate when features are roughly Gaussian. Min-max is preferred when we need a hard bounded range.

In both cases the transform parameters (min / max or μ/σ) are computed on **training data only** and then re-applied to the test/query points.

3.4 Distance with nominal attributes

For a nominal attribute, define the per-attribute difference as

$$\delta(a_j, b_j) = \begin{cases} 0 & \text{if } a_j = b_j, \\ 1 & \text{if } a_j \neq b_j. \end{cases}$$

No normalisation needed (already in $\{0, 1\}$). Example:

$$\mathbf{x}^{(1)} = \begin{bmatrix} \text{red} \\ \text{new} \end{bmatrix}, \quad \mathbf{x}^{(2)} = \begin{bmatrix} \text{blue} \\ \text{new} \end{bmatrix} \implies d_1(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}) = 1 + 0 = 1.$$

3.5 Missing values (for distance-based methods)

Principle: a missing value is treated as *maximally different* from anything.

- **Nominal:** difference is 1 if **either** value is missing; 0 only if both present and equal. Example: (red, ?) vs (blue, new) $\implies d = 1 + 1 = 2$.
- **Numeric** (normalise first): if both missing, difference = 1; if exactly one missing, different = $\max(v, 1 - v)$ where v is the present (normalised) value — i.e. push it as large as possible. Example: (0.2, ?) vs (0.7, 0.1) $\implies d = |0.2 - 0.7| + \max(0.1, 0.9) = 0.5 + 0.9 = 1.4$.

3.6 Distance-weighted k -NN

Closer neighbours should count more. Weight neighbour i by the inverse distance

$$w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}^{(i)}) + \epsilon},$$

where $\epsilon > 0$ is small and is for avoiding division by zero. Then (for regression) predict the weighted combination

$$\hat{y} = \frac{\sum_{i \in N_k(\mathbf{x})} w_i y^{(i)}}{\sum_{i \in N_k(\mathbf{x})} w_i}.$$

A variant uses **all** training points, not just k — distant ones contribute negligibly because $w_i \rightarrow 0$, at the cost of speed.

3.7 Lazy vs eager learning; complexity

k -NN is **instance-based** / **lazy**: it stores all examples and defers all computation to query time ("training = memorising"). Contrast **eager** learners (1R, decision trees, Naive Bayes, neural networks, SVM) that build a model up front.

With n training examples of dimensionality d :

- **Training time:** $O(1)$ effectively (just store) — fast.
- **Query time (one new point, brute force):** compare against all n points, each an $O(d)$ distance $\rightarrow O(nd)$.

- **Memory:** $O(nd)$ — keep the whole dataset.

So lazy methods are *fast to train*, *slow to predict*, and impractical for very large datasets without acceleration. Variants use space-partitioning structures (**KD-trees**, **ball trees**) and example-forgetting to cut both costs.

3.8 Curse of dimensionality

In high d , "nearness" loses meaning: almost all points are far apart and near the boundary. Quantitatively, for points uniform in the unit hypercube $[0, 1]^d$, a centred sub-cube of edge length s holds a volume fraction

$$\text{frac}(s) = s^d.$$

Inverting, to capture a fraction r of the data we need edge length $s = r^{1/d}$. For $d = 10$:

$$s = 0.01^{1/10} \approx 0.63 \implies \text{a sub-cube of edge } 0.63 \text{ contains only } \sim 1\% \text{ of the points,}$$

i.e. $\sim 99\%$ lie outside it (and capturing even 10% requires edge $0.1^{1/10} \approx 0.79$). Neighbourhoods that feel "local" in low dimensions are forced to span almost the whole space in high dimensions, so k -NN cannot be trusted in high d . **Remedy:** feature selection / dimensionality reduction. (High dimensionality also drives overfitting for *all* classifiers.)

3.9 Decision boundary — Voronoi tessellation

Think of each training point as a site in feature space. The training data induces a **Voronoi partition**: the Voronoi cell of training point $\mathbf{x}^{(i)}$ is

$$V(\mathbf{x}^{(i)}) = \{\mathbf{x} : d(\mathbf{x}, \mathbf{x}^{(i)}) \leq d(\mathbf{x}, \mathbf{x}^{(j)}), \forall j\},$$

i.e. the region of all points whose nearest training example is $\mathbf{x}^{(i)}$. In **1-NN**, the decision boundary is exactly the set of Voronoi edges separating cells of *different* classes — a piecewise-linear, arbitrarily shaped boundary determined by a subset of the edges.

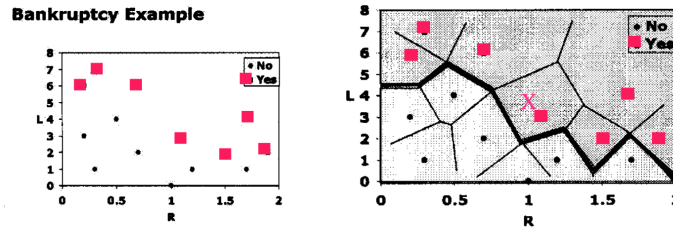


Figure 1: Bankruptcy data: raw scatter on the left. Voronoi tessellation with the class-separating boundary highlighted on the right.

3.10 Strengths and weaknesses

Strengths

- Simple: often works well in practice (like 1R and Naive Bayes).
- Naturally handles multiclass and regression; produces flexible boundaries.

Weaknesses

- Slow at prediction on large data (needs efficient indexing / variants).
- Curse of dimensionality.
- Very sensitive to **irrelevant attributes** (they inflate distances). Fixes: per-attribute weighting, feature selection.
- High-variance boundary — depends heavily on the training sample and on k ; sensitive to noise.
- **Local vs global:** standard k -NN uses only nearby points $\rightarrow$ a *local* model. 1R, decision trees, neural networks instead fit a *global* model to the whole training set.

4 Rule Learning: the 1R Algorithm

1R = "1-rule". It learns a single rule that branches on the value of **one** attribute — equivalently a **1-level decision tree** (a *decision stump*): root tests one attribute, each branch is a value, each leaf is a predicted class.

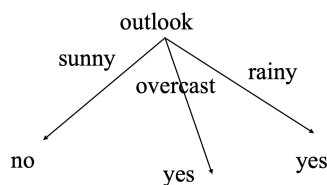


Figure 2: The outlook stump for the weather dataset.

4.1 Two design questions

1. **What class goes in each leaf?** $\rightarrow$ the **majority class** of the training examples that reach that leaf. This minimises the number of misclassified training examples *at that leaf* by construction.
2. **Which attribute (which rule) is best?** $\rightarrow$ the one whose stump makes the **fewest errors** on the training data (equivalently, highest training accuracy); break ties randomly.

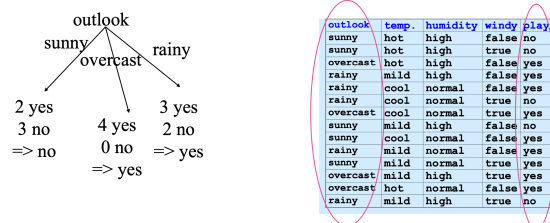


Figure 3: The *outlook* stump with per-leaf yes/no counts $\rightarrow$ majority labels.

4.2 1R on the weather data

For each attribute, build the stump (majority class per value) and count errors:

#	attribute	rules (values $\rightarrow$ class)	per-value errors	total
1	outlook	sunny $\rightarrow$ no, overcast $\rightarrow$ yes, rainy $\rightarrow$ yes	2/5, 0/4, 2/5	4/14
2	temp.	hot $\rightarrow$ no*, mild $\rightarrow$ yes, cool $\rightarrow$ yes	2/4, 2/6, 1/4	5/14
3	humidity	high $\rightarrow$ no, normal $\rightarrow$ yes	3/7, 1/7	4/14
4	windy	true $\rightarrow$ no*, false $\rightarrow$ yes	3/6, 2/8	5/14

(*random tie-break.*) Minimum total error is 4/14, achieved by **outlook** and **humidity** (a tie); choose rule 1:

```

if    outlook = sunny    then play = no
elseif outlook = overcast then play = yes
elseif outlook = rainy   then play = yes

```

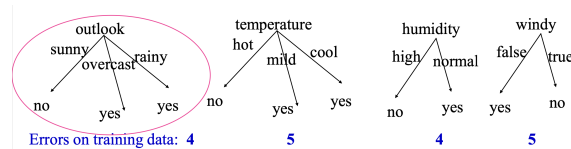


Figure 4: Four stumps with their error counts.

4.3 Pseudocode and components

Algorithm 1 1R Algorithm sketch

```

for each attribute  $A$  do
  for  $v \in A$  do
    Count how often each class appears among examples with  $v \in A$ 
    Assign  $v$  the most frequent class
  end for
  error-rate( $A$ )  $\leftarrow \frac{1}{n} \#(\text{training examples misclassified by this stump})$ 
end for
Choose the attribute  $A^*$  with the smallest error-rate

```

Viewed as a learning algorithm, 1R has three components:

- **Model class:** a rule testing a single attribute (a stump).
- **Preference (scoring) function:** number of training misclassifications.
- **Search:** exhaustive over all d attributes.

4.4 Handling missing values (generic, not just 1R)

- **Method 1: discard** instances with a missing value. Tempting but wasteful: those rows are often still useful, and if the missing attribute does not drive the decision they are as good as any.
- **Method 2: treat "missing" as its own value** (this is what 1R does). It adds an extra branch; e.g. a missing-aware *outlook* stump splits on {sunny, overcast, rainy, missing}. Assumes the *absence* of a value is itself informative.
- **Method 3: impute.** Replace ? with either (a) the most common value of that attribute over **all** training examples, or (b) the most common value of that attribute among training examples **of the same class** as the incomplete example.

4.5 Numeric attributes: discretisation

1R needs nominal attributes, so numeric ones (e.g. *temperature*, *humidity*) are **discretised** into intervals. Naive procedure:

1. **Sort** examples by the numeric attribute.
2. **Place a breakpoint** wherever the class label changes, halfway between the two values.
3. **Label** each resulting interval by its (majority) class → produces the rule.

For *temperature* this yields many tiny intervals (e.g. $<64.5 \rightarrow \text{yes}$, $[64.5, 65.5) \rightarrow \text{no}$, ...) — a **highly branching** attribute.

```
if temperature <64.5 then play=yes
elseif temperature in [64.5, 65.5) then play=no
elseif temperature in [66.5, 70.5) then play=yes
elseif temperature in [70.5, 72.5) then play=no
elseif temperature in [72.5, 77.5) then play=yes
elseif temperature in [77.5, 80.5) then play=no
elseif temperature in [80.5, 84) then play=yes
elseif temperature >=84 then play=no
```

4.6 Overfitting and a better discretisation

Overfitting: training error is tiny but test error is high — the model *memorised* the training set instead of *generalising*. The naive procedure overfits because a single noisy/mislabelled example carves out its own spurious interval that won't classify new points well.

Fix (aggregation): require each interval to contain at least a minimum number (e.g. ≥ 3) of examples of its majority class, then **merge adjacent intervals sharing the same majority class**. For *temperature* this collapses to a single useful split:

```
if temperature <= 77.5 then yes
else no*
```

(The * marks a tie broken arbitrarily; had "yes" been chosen for both sides, no breakpoint would be needed at all.)

Final rule set with numeric attributes

After the improved discretisation, re-scoring all attributes:

#	attribute	rules	per-part errors	total
1	outlook	sunny $\rightarrow$ no, overcast $\rightarrow$ yes, rainy $\rightarrow$ yes	2/5, 0/4, 2/5	4/14
2	temp.	$\leq 77.5 \rightarrow$ yes, $> 77.5 \rightarrow$ no*	3/10, 2/4	5/14
3	humidity	$\leq 82.5 \rightarrow$ yes, $(82.5, 95.5] \rightarrow$ no, $> 95.5 \rightarrow$ yes	1/7, 2/6, 0/1	3/14
4	windy	true $\rightarrow$ no, false $\rightarrow$ yes	3/6, 2/8	5/14

Now **humidity** wins with 3/14 errors — note the chosen attribute changed once numerics were handled properly.

4.7 Decision boundary of 1R

Because a binary 1R split thresholds a single attribute, the decision boundary is a **single axis-parallel hyperplane** (a vertical or horizontal line in $\mathbb{R}^2$) — far simpler than the jagged Voronoi boundary of 1-NN. This is the geometric expression of 1R's "one attribute only" bias.

4.8 1R — discussion (Holte, 1993)

- Holte (1993) found 1R's tiny rules were only **slightly less accurate** than far more complex decision trees and other state-of-art classifiers on many real datasets.
- Simple, computationally cheap, and a great baseline: *"simplicity first" pays off — always try the simple algorithms first; they give a useful first impression of the data.*

Why do simple algorithms work so well?

The structure underlying many real problems is genuinely rudimentary:

- one attribute alone may determine the class (1R's assumption);
- several attributes may contribute **independently with equal importance** (Naive Bayes like);
- a **linear combination** of attributes may suffice.

When the truth is simple, a simple, low-variance model generalises better than a flexible, high-variance one — a foreshadowing of the bias-variance tradeoff.